TP2 - Sincronização em sistemas distribuídos

BCC362 — Sistemas Distribuídos | Professor Joubert Lima | Grupos de até 3 alunos

Neste trabalho, cada grupo deverá construir uma solução distribuída para sincronizar acessos concorrentes a um recurso R hipotético. O grupo poderá implementar um algoritmo da literatura ou desenvolver uma aplicação que utilize primitivas de sincronização de uma tecnologia de mercado. No seminário, cada grupo deverá ensinar aos colegas como sua solução funciona e demonstrar sua execução concorrente.

1 Escopo e regras gerais

O foco deste TP é sincronização: exclusão mútua, ordenação dos pedidos, justiça no acesso e, nas extensões, barreiras, semáforos e locks de leitura e escrita. Neste TP2, considerar os participantes e os serviços disponíveis durante os experimentos, sem quedas ou perdas de mensagens.

Qualquer linguagem será aceita. A solução deverá usar processos independentes que se comuniquem pela rede, em contêineres, VMs ou máquinas reais. Soluções apenas multicore, apoiadas em memória compartilhada ou mutex local, não serão válidas. É permitido executar os contêineres no mesmo computador.

Na trilha de algoritmos, não será aceita uma solução em que um único coordenador fixo mantenha todas as autorizações. Na trilha de tecnologias, serão aceitas as primitivas prontas do serviço escolhido. Não basta instalar o produto ou executar um exemplo: o grupo deverá desenvolver a aplicação, explicar a primitiva e verificar sua utilização correta sob concorrência.

2 Cenário comum do Cluster Sync

Criar pelo menos 5 clientes e 5 processos no Cluster Sync, em processos ou contêineres separados. Cada cliente conhece apenas um elemento do Cluster Sync. Cada pedido tem um request_id único, por exemplo (client_id, número sequencial). O timestamp físico poderá constar no log, mas não substituirá esse identificador nem o relógio lógico exigido pelo algoritmo.

Cada cliente deverá solicitar entre 10 e 50 acessos de escrita ao recurso R. O elemento do Cluster Sync obtém autorização, aguarda a seção crítica por um sleep aleatório de 0,2 a 1 segundo, libera o acesso e responde COMMITTED. O cliente aguarda de 1 a 5 segundos aleatórios e faz o próximo pedido. Neste TP, COMMITTED confirma o término do acesso simulado; não significa persistência ou replicação de dados.

As opções de tecnologia acrescentam ao cenário o serviço de coordenação, configurado conforme os requisitos do produto. Sua implantação poderá ser mínima para a demonstração funcional. Não se exige alta disponibilidade nem implementação ou avaliação de replicação; componentes internos do produto não são o objeto deste TP.

3 Escolha e implementação

Escolher uma opção das páginas seguintes. Todos os grupos deverão demonstrar exclusão mútua sobre R; as extensões ampliam essa aplicação, sem substituir o cenário comum. Organizar o código com operações equivalentes a acquire(R, request_id) e release(R, request_id), separando a sincronização da seção crítica simulada. No TP3, essa seção poderá ser substituída pelo acesso ao Cluster Store.

4 Trilha A Algoritmos de exclusão mútua

O grupo implementará o protocolo escolhido. Bibliotecas de comunicação e serialização são permitidas; o mecanismo de sincronização não poderá ser substituído por um lock distribuído pronto. As opções ampliam o repertório do TP: são algoritmos clássicos e alternativas adicionais àquelas do enunciado anterior, não uma lista de algoritmos recém-publicados.

A1 Lamport — intermediário

Implementar REQUEST, REPLY e RELEASE, relógios lógicos e filas ordenadas por (relógio, ID do processo). Explicar as condições de entrada e a atualização das filas. Declarar e atender às hipóteses dos canais, incluindo a ordenação necessária. Comparar o custo de mensagens com Ricart–Agrawala. Referência [1].

A2 Ricart–Agrawala — intermediário

Implementar pedidos e respostas adiadas. Um processo responde imediatamente quando não disputa R ou quando o pedido recebido tem prioridade; estando na seção crítica, adia a resposta. Na saída, enviar todas as respostas adiadas. Mostrar por que receber $N-1$ respostas autoriza a entrada e medir as $2(N-1)$ mensagens por entrada na versão básica. Referência [2].

A3 Token ring — intermediário

Organizar os processos em anel lógico e permitir a entrada somente ao portador do token. Demonstrar inicialização com um único token, circulação e atendimento dos pedidos. Comparar o comportamento com pouca disputa e com todos os nós solicitando R. Explicar a justiça do acesso sob as hipóteses adotadas. Não se exige perda ou reconstrução do token.

A4 Suzuki–Kasami — intermediário a avançado

Implementar números sequenciais de pedidos, vetor de solicitações conhecidas e token contendo os últimos pedidos atendidos e a fila de pendências. Demonstrar entrada quando o token já está local e transferência quando outro processo solicita R. Mostrar como os números de sequência evitam atender novamente um pedido antigo. Referência [3].

A5 Raymond — avançado

Organizar uma árvore lógica, com ponteiros para o portador do token e filas locais de pedidos. Apresentar a rota dos pedidos e do token em uma disputa concreta. Comparar árvore balanceada e árvore em cadeia; relacionar topologia, carga e quantidade de mensagens. Não atribuir custo logarítmico a qualquer topologia. Referência [4].

A6 Maekawa — avançado

Construir conjuntos de votação com interseção par a par e limitar cada votante a uma permissão por vez. Apresentar os conjuntos e justificar a exclusão mútua. Mostrar que a versão básica pode gerar deadlock mesmo sem falhas; implementar uma variante documentada de prevenção ou resolução, com suas mensagens adicionais. Usar 7 ou 9 nós se isso facilitar uma construção adequada dos conjuntos. Referência [5].

A7 Naimi–Tréhel — avançado

Implementar exclusão mútua por token e árvore lógica dinâmica baseada em reversão de caminhos. Mostrar como os pedidos alteram os ponteiros e chegam ao portador do token. Comparar o tráfego com Raymond ou token ring, distinguindo custo médio de pior caso e declarando as hipóteses da análise. Referência [6].

Em todas as opções, serializar ou enfileirar corretamente pedidos recebidos pelo mesmo processo. Um mutex local pode proteger estruturas internas do nó; ele não substitui o protocolo que coordena os nós do Cluster Sync.

5 Trilha B Tecnologias de sincronização

O grupo desenvolverá os clientes e o Cluster Sync usando o serviço escolhido. Deverá explicar o significado da primitiva utilizada, o estado que ela mantém e a sequência de uma disputa. A instalação terá apenas o detalhamento necessário à reprodução; o funcionamento da sincronização será o centro da apresentação.

B1 ZooKeeper com API direta — intermediário

Implementar mutex com znodes efêmeros sequenciais. Cada concorrente cria seu znode e identifica sua posição; quem possui o menor número entra, e os demais observam o predecessor. Após a notificação, reavaliar a condição antes de entrar. Mostrar criação, espera, aquisição e remoção do znode na liberação normal. Explicar por que observar o predecessor evita acordar todos os concorrentes. Referência [7].

B2 ZooKeeper com Apache Curator — intermediário

Em Java, usar InterProcessMutex para o cenário de R e acrescentar uma primitiva: DistributedDoubleBarrier, InterProcessSemaphoreV2 ou InterProcessReadWriteLock. Explicar a receita que está por trás da chamada acquire(), sem limitar a apresentação à API. B1 e B2 podem ser escolhidas por grupos distintos, pois uma implementa a receita e a outra compara primitivas prontas. Referência [8].

B3 etcd — intermediário

Usar a API de concorrência, sessões e mutex para coordenar os 5 processos do Cluster Sync. Mostrar o estado criado para cada disputa, a ordem de espera e a liberação normal. Explicar o papel das revisões e das leases na API utilizada, mantendo as sessões válidas durante os experimentos. Medir espera e concorrência; não se exige testar expiração, eleição ou replicação. Referência [9].

B4 Consul — intermediário

Usar sessions e aquisição/liberação de uma chave KV para o mutex de R. Mostrar como apenas um concorrente obtém a autorização e como os demais aguardam ou tentam novamente. Explicar que o lock é cooperativo: todos os processos da aplicação precisam respeitá-lo. Manter as sessões válidas e comparar uma espera baseada em consulta com a abordagem por notificações do ZooKeeper. Referência [10].

B5 Hazelcast — intermediário

Utilizar FencedLock com o CP Subsystem explicitamente habilitado, seguindo a configuração mínima exigida pelo produto. Mostrar disputa, aquisição, liberação e reentrância, quando utilizada. Acrescentar um semáforo distribuído para uma etapa de capacidade limitada. Explicar a diferença entre mutex e semáforo; não se exige estudar falhas, fencing de recurso externo ou o protocolo interno de replicação. Referência [11].

Como equilibrar os trabalhos das duas trilhas

Algoritmos próprios exigem implementação e explicação das mensagens e dos estados. APIs prontas reduzem esse esforço; por isso, os grupos de B2 a B5 deverão acrescentar uma extensão de sincronização, além do mutex sobre R. B1 já exige a implementação da receita de lock. Em todas as opções, apresentar logs e uma verificação automática da propriedade esperada.

Fixar as versões utilizadas. Antes de escolher Hazelcast, conferir os requisitos de edição e licença da versão disponível. Docker e Kubernetes podem hospedar a aplicação; sua instalação, por si só, não constitui uma solução de sincronização para este TP.

6 Mensageria e outras primitivas

B6 Ordenação de acessos por mensageria — avançado

Implementar ACQUIRE e RELEASE e uma fila privada de pedidos em cada processo do Cluster Sync. Todos devem observar uma sequência compatível dos eventos de R, remover pedidos na liberação e autorizar somente o processo correspondente ao primeiro pedido pendente. Cada mensagem deverá identificar processo, recurso e request_id. O broker transporta e ordena eventos; os processos implementam a regra de acesso.

Kafka: colocar os eventos de R em uma única partição e usar um grupo de consumidores distinto para cada processo do Cluster Sync, para que todos leiam a sequência inteira. A ordem é por partição. Iniciar todos de uma posição conhecida e garantir que estejam prontos antes da carga. Não se exige replay para recuperação ou implementação de consenso. Referência [12].

Redis Pub/Sub: usar um único canal de uma instância, com todos os assinantes conectados antes da carga; justificar a ordem observada e garantir que todos recebam os eventos no experimento. A semântica é de entrega no máximo uma vez: o escopo assume ausência de desconexões. Não confundir Pub/Sub com Redlock ou com um lock implementado por SET NX. Referência [14].

RabbitMQ ou Mosquitto: o grupo poderá propor sua utilização, mas deverá demonstrar a mesma sequência em todos os participantes. Consumidores competindo numa única fila RabbitMQ dividem mensagens; são necessárias filas próprias para disseminar eventos a todos. FIFO de um canal isolado não demonstra ordem total entre vários publicadores. Se necessário, implementar uma camada de ordenação e justificar seu algoritmo. Referência [13].

Extensões que aprofundam a sincronização

Extensão	Implementação e propriedade a verificar
Barreira por rodadas	Os 5 processos chegam à barreira; nenhum inicia a etapa seguinte antes da chegada de todos. Identificar a rodada para evitar misturar mensagens de fases diferentes.
Barreira dupla	Sincronizar o início e o fim de uma atividade. Nenhum participante avança para a próxima rodada enquanto houver alguém executando a rodada atual.
Semáforo distribuído	Usar K=2 e K=3 permissões em uma etapa separada. Verificar que a quantidade de acessos simultâneos nunca ultrapassa K e que há uso concorrente das permissões.
Leitores e escritores	Permitir várias leituras simultâneas, mas escrita exclusiva. Variar a proporção de leitores e escritores e avaliar prioridade e starvation.

Múltiplos recursos	Usar R1 e R2 e pedidos que precisam de ambos. Construir uma disputa com risco de deadlock e evitá-lo por uma ordem global de aquisição.

Essas extensões podem ser implementadas pelo grupo ou com primitivas do serviço escolhido. São desafios de sincronização em operação normal: não incluem queda de participante, perda de token, failover ou recuperação. A exclusão mútua sobre R continua obrigatória para preservar a continuidade com o TP3.

7 Experimentos e evidências de sincronização

Verificação da seção crítica simulada

Registrar REQUEST, ACQUIRE, ENTER, EXIT, RELEASE e COMMITTED, com request_id e ID do processo. Incluir relógio lógico, posição do token, fila ou revisão quando fizer parte do mecanismo. A apresentação deverá mostrar por que um determinado processo entrou e por que os demais esperaram.

Usar um observador que receba BEGIN e END de todos os processos e mantenha a contagem de acessos ativos a R. A aplicação aguarda o registro de BEGIN antes do sleep e registra END depois dele. O observador apenas registra e acusa sobreposições; não concede permissão, não bloqueia BEGIN de outro processo e não funciona como mutex. Essa verificação dispensa comparar horários físicos de máquinas diferentes.

Realizar uma execução com a proteção DE SINCRONIZAÇÃO desabilitada, para mostrar que a instrumentação detecta acessos simultâneos. Em seguida, repetir com o protocolo habilitado. Um contador atualizado atomicamente pelo observador pode medir ocupação; não pode ser usado como mecanismo de autorização pelos processos.

Experimento	**O que executar e avaliar**
Carga normal	5 clientes, 10 a 50 acessos por cliente, com os intervalos do cenário comum. Conferir ausência de sobreposição e conclusão de todos os pedidos.
Alta contenção	Retirar o sleep de 1 a 5 segundos entre pedidos, preservando a duração da seção crítica. Acionar os clientes conjuntamente para produzir disputa real.
Carga desigual	Um cliente solicita R com frequência maior. Mostrar a distribuição de acessos e a espera dos demais; discutir justiça e prioridade.
Quantidade de participantes	Comparar 5 e 10 processos do Cluster Sync, quando viável. Relacionar espera e tráfego ao crescimento do número de participantes.
Extensão escolhida	Aplicar a verificação correspondente: limite K no semáforo, chegadas por rodada na barreira ou compatibilidade entre leitores e escritores.

Métricas e interpretação

Executar pelo menos 3 repetições da carga normal e da alta contenção. Informar número de pedidos concluídos, tempo total, espera mediana para aquisição, acessos por cliente e mensagens por entrada, quando instrumentáveis. Registrar configuração, duração da seção crítica e sementes aleatórias.

Comparar o custo teórico com o tráfego observado, separando mensagens do protocolo de mensagens de instrumentação. Não comparar diretamente resultados com durações de seção crítica ou cargas diferentes. Medir a espera no próprio processo com relógio monotônico, evitando subtrair timestamps de hosts distintos.

Explicar exclusão mútua, progresso, deadlock e starvation conforme as garantias da opção escolhida. Ausência de starvation numa execução curta é uma observação experimental, não uma prova. O trabalho não incluirá cenários de falha ou avaliação de replicação.

8 Entrega e seminário da disciplina

Material a entregar

Entregar código-fonte, README com instalação e execução, dependências e versões, configuração dos processos ou contêineres, scripts de carga, logs selecionados e verificador das propriedades. Anexar apresentação de 10 a 15 slides, sem contar referências e logs, com cenário, mecanismo de sincronização, arquitetura, resultados e discussão. Não incluir credenciais no repositório.

Apresentação sugerida de 25 minutos

Etapa	Tempo	Conteúdo
Problema e primitiva	3 min	O que será sincronizado e qual propriedade se deseja garantir.
Funcionamento interno	6 min	Estados, mensagens, token, votação, fila ou receita do serviço; exemplo concreto de disputa.
Demonstração	8 min	Acesso concorrente a R, instrumentação e extensão, quando aplicável.
Resultados	3 min	Espera, custo de mensagens, justiça e limitações do mecanismo.
Discussão	5 min	Perguntas dos colegas e do professor; participação de todos os integrantes.

Critérios sugeridos de avaliação

Critério	Peso
Correção da sincronização e evidências das propriedades	30%
Domínio do algoritmo ou da primitiva e explicação interna	25%
Experimentos de concorrência e interpretação dos resultados	20%
Reprodutibilidade, qualidade do código e instrumentação	15%
Apresentação, participação e respostas às perguntas	10%

Escolha e distribuição dos temas

Evitar repetir a mesma opção com o mesmo desafio. É possível que dois grupos utilizem ZooKeeper, por exemplo, desde que um implemente a receita de mutex e outro explore mutex com barreira dupla ou semáforo. Reutilizar a infraestrutura do TP1 é permitido; neste TP, a instalação não substitui a demonstração de sincronização.

Para uma turma com 8 grupos, sugere-se: Ricart–Agrawala; Suzuki–Kasami; Raymond ou Naimi–Tréhel; Maekawa; ZooKeeper direto; ZooKeeper/Curator com barreira dupla; etcd com uma extensão; e mensageria com ordenação. Consul e Hazelcast são alternativas conforme a infraestrutura disponível.

9 Referências para estudo e implementação

Os artigos fundamentam as propriedades dos algoritmos; a documentação oficial orienta APIs e receitas. Consultar a versão efetivamente utilizada. Referências consultadas em 13 de outubro de 2026.

[1] Lamport, 1978. Time, Clocks, and the Ordering of Events in a Distributed System.
https://lamport.azurewebsites.net/pubs/time-clocks.pdf

[2] Ricart e Agrawala, 1981. An Optimal Algorithm for Mutual Exclusion in Computer Networks.
https://doi.org/10.1145/358527.358537

[3] Suzuki e Kasami, 1985. A Distributed Mutual Exclusion Algorithm.
https://doi.org/10.1145/6110.214406

[4] Raymond, 1989. A Tree-Based Algorithm for Distributed Mutual Exclusion.
https://doi.org/10.1109/12.24268

[5] Maekawa, 1985. A √N Algorithm for Mutual Exclusion in Decentralized Systems.
https://doi.org/10.1145/214438.214445

[6] Naimi, Tréhel e Arnold, 1996. A Log(N) Distributed Mutual Exclusion Algorithm Based on Path Reversal.
https://doi.org/10.1006/jpdc.1996.0041

[7] Apache ZooKeeper. Recipes and Solutions.
https://zookeeper.apache.org/doc/r3.9.2/recipes.html

[8] Apache Curator. Recipes.
https://curator.apache.org/docs/recipes/

[9] etcd. API e primitivas de concorrência.
https://etcd.io/docs/v3.6/learning/api/

[10] HashiCorp Consul. Sessions and distributed locks overview.
https://developer.hashicorp.com/consul/docs/automate/session

[11] Hazelcast. FencedLock e configuração do CP Subsystem.
https://docs.hazelcast.com/hazelcast/5.7/cp-subsystem/configuration

[12] Apache Kafka. Introduction e garantias de ordenação por partição.
https://kafka.apache.org/documentation/

[13] RabbitMQ. Queues e ordenação de mensagens.
https://www.rabbitmq.com/docs/queues#message-ordering

[14] Redis. Pub/Sub e semântica de entrega.
https://redis.io/docs/latest/develop/pubsub/